

A Challenge-Bound Memory-Consistency Predicate for Model-Key Release

Imran Siddique

OPAQUE Systems

October 1, 2026

Technical report, version 1

Scope of this edition. This is an author-prepared technical report, not peer reviewed. It revises a manuscript dated September 4, 2026. Every result reported here is software-only: repository unit tests and separate review probes, run on one Windows workstation against a pinned source revision on September 4, September 27 and October 1, 2026 (the last in a fresh environment), and compared with the repository's main branch of September 30, 2026. The 256 MiB protected-guest figure in Section 5 comes from a receipt retained in the repository from August 27, 2026; that hardware run was not repeated. No hardware quote chain, physical attack or production deployment was exercised.

Abstract

A key broker that gates model-key release on attestation checks code identity. It does not see how a designated memory region behaves while the release request is in flight. We describe a supplementary predicate implemented in Weight Custody Manifest. A runtime derives page-specific values and two traversal orders from a secret and the broker's challenge, writes every logical page, reads every page back, and signs a transcript of declared geometry, page counts, a readback commitment and a mismatch flag. The broker accepts the transcript only if it carries the current challenge, reports no mismatch and verifies under a pinned key, in addition to its existing release policy. At a pinned revision, 35 local tests across the memory sweep and broker modules passed on September 4, September 27 and October 1, 2026. A controlled aliased adapter yields an authentic negative transcript, which shows why signature verification and release approval have to be separate checks. We also analyze a retained 256 MiB protected-guest receipt without treating it as a fresh end-to-end key-release experiment. The predicate gives a bounded observation over a declared logical range. It does not prove full model-memory coverage, defeat a physical-owner attack or establish that memory stays unchanged after the check.

1 Scope and prior mechanisms

Remote memory attestation and challenge-driven traversal are established techniques. SWATT checks embedded-device memory with a challenge-seeded pseudorandom traversal [1]. Proofs of secure erasure fill a device's memory with verifier-chosen data and check the response [2]. Writing a distinct value to every address and reading it back is also the classic test for address-decoder faults, where two addresses reach one cell [3]. Data attestation in memory devices appears in patent literature [4], and the Confidential Containers Trustee architecture already evaluates evidence before releasing secrets [5]. What we add is one more release predicate that composes these pieces at a model-key broker. We claim neither memory checking nor attestation-gated key release.

The motivating requirement is a custodian's request for a fresh, authenticated report that a selected scratch range behaved consistently under a write/read challenge. The range may be intended for later protected computation, but the current broker does not prove that association. A small successful scratch test says nothing about an entire model allocation.

2 Actors and assumptions

The broker holds a model key and a release policy. A runtime owns a designated memory adapter, a sweep secret and a signing key. The broker pins the expected signing public key out of band. The adapter exposes page size, page count, `write_page` and `read_page`. The software reference uses a bytearray; a historical guest tool uses an anonymous mmap region and records whether memory locking succeeded.

We assume honest sweep code, correct adapter semantics, protected randomness and signing-key custody, trustworthy broker policy, sound cryptographic primitives and a fresh broker challenge that is consumed on use. A check that the secret is at least 32 bytes long is not proof of entropy. A public-key pin does not by itself establish that the key is held by the attested runtime. That binding needs a provisioning or attestation protocol outside this sweep helper.

The operation overwrites the selected range, so it must run on a scratch region or before live model data is placed there. No preservation protocol for existing content is implemented. The interface also has no explicit architecture-specific memory-barrier primitive.

3 Sweep construction

For secret s , nonce n and page index i , compute HMAC-SHA256 over n followed by i as unsigned 64-bit big-endian bytes, and expand the result with SHAKE256 to one page. This gives a page-specific test value that cannot be predicted without the secret.

For each domain label, write or read, derive another HMAC output. Its two halves choose a start and a step modulo the page count N ; the step is incremented until it is coprime to N . Indices are enumerated as $(\text{start} + k \cdot \text{step}) \bmod N$ for k from zero to $N-1$. Coprimality guarantees that each logical index is visited exactly once. Domain separation does not guarantee that the write and read orders differ on every input.

The runtime writes the expected value to every logical page before the read pass begins. It compares each page read back with its expected value and accumulates a readback hash over the read index and the SHA-256 of the bytes actually read. Any failed comparison sets the mismatch flag. The implementation calls this field `aliasing_detected`, although corruption or inconsistent adapter behavior can also produce a mismatch.

The transcript names the algorithm `wcm-full-range-v1` and records the challenge, range start, range length, page size, pages written and read, readback hash, mismatch flag and signer public key. Ed25519 signs its canonical representation. The written and read counts are filled in from the declared page count when the transcript is built; the passes do not count them. Both passes enumerate a full permutation and an adapter error aborts the sweep, so for a completed sweep the counts are correct, but a verifier should read them as declared values. Traversal needs constant auxiliary state, page processing needs $O(B)$ transient memory for page size B , and total data work is $O(NB)$. These follow from the construction and were not measured.

4 Broker evaluation and security boundary

The transcript verifier checks the algorithm name, consistency of geometry and counts, the pinned key and the signature. It correctly verifies a signed negative finding. It does not receive the current broker challenge and therefore does not establish request freshness. The broker separately compares the transcript nonce with the release challenge and rejects any transcript with the mismatch flag set. Its release path consumes the challenge and requires every other configured check to pass.

The memory predicate applies only when the release policy requires it. Missing evidence or an absent verifier key then causes denial in the assured configuration. An explicit legacy declarative mode can bypass signed-sweep evaluation and is outside that configuration. Channel-bound sealing is another optional broker feature; a default release returns the key unsealed.

A coherent self-declared range is not an independently authorized range. The broker has no expected-extent pin for this predicate, at the pinned revision or on main. It also does not prove physical uniqueness, prevent remapping after the check, explain the cause of a mismatch or observe an external DDR interposer. BadRAM shows practical static memory aliasing against trusted execution environments [6]. Battering RAM uses a

low-cost interposer to switch aliasing on at runtime, after boot-time alias checks have passed [7]. A sweep at release time has the same exposure, because it observes the range only while it runs. Our tests establish no defense against either attack.

5 Evaluation

Source revision: `weight-custody-manifest 2acedfa883204f7d1fd457d3ee603198d85be82c` [8]. The selected modules, `test_memory_sweep.py` and `test_kbs.py`, hold 35 tests, and all 35 passed on September 4, 2026. On September 27 and again on October 1, 2026, the same modules ran against a git archive of that commit, whose embedded commit identifier was checked before extraction, and all 35 passed both times. The October 1 run used fresh clones and a new virtual environment with Python 3.12.10, pytest 8.4.2 and cryptography 50.0.0.

The cases cover the sweep and its use at the broker. A small real bytearray range passes. A controlled adapter that maps two logical pages to one backing slot produces a mismatch. An altered page count, a flipped mismatch flag and an unauthorized signing key all fail verification. Broker tests exercise required-but-absent evidence, a mismatch-positive transcript, clean signed release, and host-signed or replayed transcripts. These are software cases with reference providers. They do not establish hardware quote-chain verification in a production deployment.

Separate probes confirm two boundaries. An authentic negative transcript passes signature verification. A transcript signed for an old challenge also passes the helper, because freshness is the broker's job. Both probes gave the same result on all three dates at the pinned revision, on September 27 at revision 94d5519 (September 26, 2026), and on October 1 at main 5ef8c12 (September 30, 2026). The signature algorithm behaves as specified in both cases; the probes show why the broker needs its own challenge and mismatch checks. Commands, source commit identifiers, package versions and logs accompany the paper.

On main, the sweep module has no commits since the pinned revision and the broker's memory check is unchanged. The broker module changed in six merged revisions in between, and two of them matter for a deployment of this predicate. One lets a release policy require the AMD SEV-SNP boot-time alias-check bit carried in the attestation report, a hardware statement that complements the runtime sweep and shares its time-of-check limit [7]. The other makes cryptographic CPU quote verification the default unless the manifest explicitly waives it. In the same two test modules on main, 46 tests passed on October 1, 2026; the 11 added tests are broker tests and the memory tests are unchanged. Apart from the probes and these source facts, the statements in this report describe the pinned revision.

The repository retains an August 27, 2026 receipt reporting a 268,435,456-byte sweep: 65,536 pages of 4,096 bytes in 2.178 seconds inside a protected guest, with locking reported successful. We did not rerun that hardware experiment. The repository's offline re-check of the receipt (13 tests) passed on October 1 at the pinned revision; it checks the recorded fields and the custody record chain. The receipt stores the sweep outcome as fields and does not include the signed transcript, so the sweep signature cannot be re-verified from it. It records nonce derivation as SHA-256 of an HCL report and omits the raw report, so the digest does not independently prove a fresh broker challenge. It also does not join the sweep, attested key provisioning and broker release into one independently verifiable execution.

6 Engineering implications

A stronger deployment should pin the expected range geometry and its relationship to the model allocation in the release request, authenticate the sweep key through the platform's own evidence, define cache and barrier semantics, and close the interval between test and use. Each is a separate design or integration task. None of them is implemented merely because the current transcript has a range field and a signature.

7 Limitations and next experiments

The construction observes a designated logical range under a trusted adapter. It cannot authenticate reality against a dishonest signer, resist physical attack or guarantee future memory behavior. The next experiments should carry an unpredictable broker nonce through a real protected-runtime sweep and a verified quote path,

retain the signed transcript, reject wrong-range and old-transcript substitutions, and have the captured evidence verified by someone other than the author. Hardware-specific ordering and mapping claims need their own tests. What this report supports is one bounded additional release condition for a model-key broker.

References

- [1] A. Seshadri, A. Perrig, L. van Doorn and P. Khosla. SWATT: SoftWare-based ATTestation for Embedded Devices. IEEE Symposium on Security and Privacy, 2004. <https://users.ece.cmu.edu/~adrian/projects/swatt.pdf>.
- [2] D. Perito and G. Tsudik. Secure Code Update for Embedded Devices via Proofs of Secure Erasure. ESORICS 2010, Lecture Notes in Computer Science 6345, Springer, 2010. https://doi.org/10.1007/978-3-642-15497-3_39.
- [3] A. J. van de Goor. Testing Semiconductor Memories: Theory and Practice. John Wiley and Sons, 1991.
- [4] A. Troia and A. Mondello (Micron Technology). Data attestation in memory. US Patent 11,163,912 B2, granted November 2, 2021. <https://patents.google.com/patent/US11163912B2/en>.
- [5] Confidential Containers. Attestation with Trustee. Project documentation, accessed October 1, 2026. <https://confidentialcontainers.org/docs/attestation/>.
- [6] J. De Meulemeester, L. Wilke, D. Oswald, T. Eisenbarth, I. Verbauwhede and J. Van Bulck. BadRAM: Practical Memory Aliasing Attacks on Trusted Execution Environments. 46th IEEE Symposium on Security and Privacy, 2025. <https://badram.eu/>.
- [7] J. De Meulemeester, D. Oswald, I. Verbauwhede and J. Van Bulck. Battering RAM: Low-Cost Interposer Attacks on Confidential Computing via Dynamic Memory Aliasing. 47th IEEE Symposium on Security and Privacy, 2026. <https://batteringram.eu/>.
- [8] Weight Custody Manifest source, pinned revision 2acedfa883204f7d1fd457d3ee603198d85be82c. <https://github.com/agentrust-io/weight-custody-manifest/tree/2acedfa883204f7d1fd457d3ee603198d85be82c>.